

Rôle et but de chaque composant

Projet UserMgmt — plateforme de gestion d'utilisateurs et de dépôt de mémoires universitaires.

Backend	ASP.NET Core 8 · Entity Framework Core · Serilog
Frontend	Angular (composants standalone) · keycloak-angular
Identité	Keycloak 26 — realm <code>usermgmt</code>
Données	PostgreSQL · Redis · MinIO (S3)
Observabilité	Prometheus · Grafana
Entrée réseau	Gateway nginx, port 80

Document établi à partir du code de la branche `main`. Les schémas d'ensemble et le déroulé d'une session figurent dans `architecture.html`.

1 Le principe directeur

Deux décisions structurent tout le reste, et expliquent le rôle de la plupart des composants.

Keycloak détient les identités, l'application détient les profils

Rôle. L'authentification, les mots de passe et les rôles vivent exclusivement dans Keycloak. La table `Users` de l'application ne conserve que des données de profil : prénom, nom, téléphone, service.

Pourquoi. L'API n'a jamais à manipuler de mot de passe ni à émettre de jeton — elle se contente de vérifier une signature. C'est aussi ce qui permet d'ajouter plus tard un SSO ou un fournisseur externe sans toucher au code applicatif. Le revers : deux sources de vérité à réconcilier, d'où le middleware de provisionnement décrit en section 3.

Un point d'entrée unique : la gateway

Rôle. Le navigateur ne joint qu'un seul port, le 80. La gateway répartit ensuite selon le préfixe d'URL vers le frontend, l'API, Keycloak, Grafana ou MinIO.

Pourquoi. Frontend, API et Keycloak partagent ainsi la même origine : aucun CORS à configurer, aucune URL absolue dans le frontend, et un seul port à exposer en production. En contrepartie, Keycloak émet ses jetons avec l'adresse publique alors que l'API le joint par le réseau interne — le décalage que traite

`InternalJwksConfigurationRetriever`.

2 Les conteneurs

Les dix services déclarés dans `docker-compose.yml`, tous sur le réseau `usermgmt-network`.

gateway

NGINX · :80

Rôle. Reverse proxy et unique porte d'entrée. Route `/` vers le frontend, `/api/` vers l'API, `/auth/` vers Keycloak, `/grafana/` et `/minio/` vers les interfaces d'administration.

Pourquoi. Unifie l'origine, supprime le besoin de CORS et masque la topologie interne. C'est le seul service dont le port est réellement destiné à être publié.

web

NGINX · BUNDLE ANGULAR

Rôle. Sert les fichiers statiques produits par `ng build`.

Pourquoi. Un frontend compilé n'a pas besoin de runtime Node : une image nginx suffit, l'empreinte est minimale et le déploiement immuable.

api

ASP.NET CORE 8 · :8080

Rôle. Cœur métier. Expose l'API REST, valide les jetons, applique les règles d'autorisation, parle à la base, au stockage objet et à l'API d'administration de Keycloak. Expose aussi `/metrics` et `/health`.

Pourquoi. C'est le seul composant qui décide : le frontend n'est qu'une vue, et les contrôles de rôles côté navigateur ne sont là que pour le confort d'affichage.

keycloak

KEYCLOAK 26 · REALM USERMGMT

Rôle. Fournisseur d'identité OpenID Connect. Détient les comptes, les mots de passe et les trois rôles applicatifs (`User`, `Admin`, `SuperAdmin`) ; émet les jetons d'accès. Le realm est importé au démarrage depuis `keycloak/realm-export.json`.

Pourquoi. Déléguer l'authentification à un produit éprouvé évite d'écrire soi-même hachage de mots de passe, gestion de sessions, rotation de clés et flux OAuth — la partie la plus facile à rater en matière de sécurité.

keycloak-db

POSTGRESQL 16

Rôle. Base dédiée à Keycloak (comptes, sessions, configuration du realm).

Pourquoi. Séparée de la base applicative pour que les identités et les mots de passe ne se trouvent jamais dans `usermgmt` : une sauvegarde, une restauration ou un accès en lecture sur la base métier n'expose aucun secret d'authentification.

postgres

POSTGRESQL 16 · BASE USERMGMT

Rôle. Base métier : profils utilisateurs et mémoires. Le schéma est géré par les migrations EF, appliquées automatiquement au démarrage de l'API.

Pourquoi. Les données relationnelles du domaine (un mémoire appartient à un utilisateur, un statut, une année) et les requêtes de recherche paginée s'y prêtent naturellement.

minio

STOCKAGE OBJET COMPATIBLE S3

Rôle. Conserve les PDF des mémoires dans le bucket `usermgmt`, en objets privés.

Pourquoi. Les fichiers binaires n'ont pas leur place dans PostgreSQL : ils alourdissent les sauvegardes et les transactions. MinIO parle le protocole S3, ce qui permet de basculer vers un stockage cloud sans changer le code. Les objets restant privés, tout téléchargement passe par l'API, qui peut donc contrôler les droits.

redis

REDIS 7

Rôle. Aujourd'hui limité à un usage : après chaque dépôt, `StorageService` écrit une clé `file:{objet}` avec une durée de vie de six heures.

Pourquoi. Le service est en place pour accueillir du cache et des sessions, mais aucune lecture n'exploite encore cette clé — l'écriture est un marqueur sans consommateur. À considérer comme une base posée pour la suite plutôt que comme un composant actif.

prometheus

COLLECTE · 15 S

Rôle. Interroge `api:8080/metrics` toutes les quinze secondes sous le nom de tâche `aspnetcore`, et conserve les séries temporelles.

Pourquoi. Donne l'historique nécessaire pour distinguer une lenteur ponctuelle d'une dégradation installée — ce qu'un simple log ne permet pas.

grafana

VISUALISATION · :3000

Rôle. Interroge Prometheus et affiche deux tableaux de bord provisionnés automatiquement : l'un centré sur le trafic HTTP, l'autre sur les compteurs métier et la latence au 95^e centile.

Pourquoi. Rend les métriques exploitables sans écrire de PromQL. Le provisionnement par fichiers garantit que tout le monde voit les mêmes tableaux après un simple `docker compose up`.

3 Le backend, dossier par dossier

Program.cs

COMPOSITION

Rôle. Point d'entrée unique : configure Serilog, EF Core, la validation des jetons, CORS, Swagger, les métriques, puis assemble le pipeline HTTP et applique les migrations au démarrage.

Pourquoi. L'ordre du pipeline est signifiant — l'authentification précède l'autorisation, elle-même suivie du provisionnement de profil. Tout est visible au même endroit.

Infrastructure/InternalJwksConfigurationRetriever.cs

SÉCURITÉ

Rôle. Récupère le document de découverte OpenID et les clés publiques de Keycloak *par le réseau interne*, en ignorant l'adresse publique annoncée dans le `jwks_uri`.

Pourquoi. Keycloak annonce ses URLs avec son nom d'hôte public (`http://localhost/auth`), injoignable depuis le conteneur de l'API où `localhost` désigne l'API elle-même. Sans cette pièce, aucune clé n'est récupérée et donc aucune signature ne peut être vérifiée. C'est ce blocage qui avait initialement conduit à désactiver toute la validation.

Infrastructure/AppDbContext.cs

EF CORE

Rôle. Contexte Entity Framework : déclare les tables `Users` et `Memoires`, leurs relations et leurs contraintes.

Pourquoi. Centralise la traduction entre objets C# et schéma relationnel, et sert de référence aux migrations.

Models/User.cs · Models/Memoire.cs

DOMAINE

Rôle. `User` est un profil local, sans champ d'authentification depuis la migration `RemoveAuthFieldsFromUser`.

`Memoire` porte le titre, l'auteur, l'année, la spécialité, le statut, la note de rejet et `FileUrl`, l'adresse de consultation du PDF.

Pourquoi. Le retrait des colonnes d'authentification matérialise la frontière avec Keycloak : il devient impossible de stocker un mot de passe par accident.

Controllers/UsersController.cs

8 ROUTES

Rôle. Consultation et mise à jour de son propre profil (`/me`), puis administration : liste paginée avec recherche et filtres, création, modification, suppression, statistiques. Les routes d'administration exigent `Admin` ou `SuperAdmin`.

Pourquoi. Toute écriture sur un compte doit se répercuter dans Keycloak, pas seulement en base — d'où la délégation systématique à `KeycloakAdminService`.

Controllers/MemoiresController.cs

8 ROUTES

Rôle. Dépôt d'un mémoire avec son PDF, consultation de ses propres dépôts, liste complète pour les administrateurs, téléchargement du fichier, validation, rejet motivé et suppression. Incrémente au passage cinq compteurs Prometheus.

Pourquoi. Le téléchargement passe par l'API plutôt que par une URL MinIO directe : c'est ce qui permet de vérifier les droits et de compter les consultations.

Services/KeycloakAdminService.cs

CLIENT HTTP

Rôle. Client de l'API d'administration de Keycloak : crée les comptes, assigne les rôles, active ou désactive, supprime, et lit le statut réel d'un utilisateur. S'authentifie en `client_credentials` avec mise en cache du jeton.

Pourquoi. Sans lui, l'application afficherait des rôles issus de sa propre base, qui pourraient contredire ceux de Keycloak. Le compte de service a besoin des rôles `manage-users`, `view-users` et `view-realm` — sans eux, tous les appels échouent en 403.

Services/StorageService.cs

S3 + REDIS

Rôle. Dépose les PDF dans MinIO en objets privés sous une clé unique, crée le bucket s'il manque, puis renvoie une URL interne `/api/memoires/file?key=...`. Fournit aussi le téléchargement et le test d'existence.

Pourquoi. Isole toute la connaissance du stockage derrière une interface : les contrôleurs ignorent qu'il s'agit de S3. Renvoyer une URL d'API plutôt qu'un lien signé garde le contrôle d'accès côté serveur.

Services/UserProfileService.cs

RÉCONCILIATION

Rôle. `EnsureProfileAsync` crée le profil local à partir des claims du jeton s'il n'existe pas encore.

Pourquoi. Un compte peut naître dans Keycloak sans passer par l'API — inscription directe, import, SSO. Créer le profil à la première requête évite d'avoir à synchroniser les deux systèmes.

Middleware/UserProfileProvisioningMiddleware.cs

PIPELINE

Rôle. Sur chaque requête authentifiée, appelle `EnsureProfileAsync` avant que le contrôleur ne s'exécute.

Pourquoi. Placé après l'authentification, il garantit à tous les contrôleurs qu'un profil existe. Aucun d'eux n'a donc à gérer le cas « utilisateur authentifié mais inconnu en base ».

Extensions/ClaimsPrincipalExtensions.cs

UTILITAIRE

Rôle. `GetCurrentUserId` extrait l'identifiant depuis le claim `sub` ; si ce n'est pas un GUID, en dérive un identifiant stable.

Pourquoi. Rend la clé primaire locale indépendante du format d'identifiant choisi par le fournisseur d'identité.

Migrations/

4 MIGRATIONS

Rôle. Historique du schéma : création initiale, ajout des mémoires, retrait des champs d'authentification, ajout de `FileUrl`. Appliquées au démarrage par `Database.Migrate()`.

Pourquoi. Le schéma se reconstruit à l'identique sur n'importe quel poste, sans script SQL à jouer à la main.

4 Le frontend

app.config.ts

AMORÇAGE

Rôle. Initialise keycloak-angular en `check-sso` avec PKCE S256, enregistre l'intercepteur et les routes.

Pourquoi. Le mode `check-sso` vérifie discrètement une session existante sans imposer l'écran de connexion : un utilisateur déjà authentifié arrive directement dans l'application.

core/guards/auth.guard.ts

AUTHGUARD · GUESTGUARD

Rôle. `AuthGuard` redirige vers Keycloak si la session manque, puis compare les rôles requis par la route à ceux du jeton. `GuestGuard` renvoie vers le tableau de bord un utilisateur déjà connecté qui tenterait d'accéder à la page de connexion.

Pourquoi. Évite d'afficher un écran vide suivi d'une erreur 403. C'est un filtre de navigation, pas une mesure de sécurité : la décision qui compte reste celle de l'API.

core/interceptors/auth.interceptor.ts

HTTP

Rôle. Ajoute l'en-tête `Authorization: Bearer` aux seules requêtes dont le chemin commence par `/api`, en récupérant un jeton rafraîchi.

Pourquoi. Restreindre au préfixe `/api` évite de transmettre le jeton à Keycloak lui-même ou à des ressources statiques. Le rafraîchissement automatique dispense chaque service de gérer l'expiration.

core/services/auth.service.ts

ÉTAT DE SESSION

Rôle. Traduit les événements de keycloak-angular en signaux Angular (`isLoggedIn`, `isAdmin`, `isSuperAdmin`, `currentUser`) et charge le profil dès l'authentification.

Pourquoi. Les composants n'interrogent jamais Keycloak directement : ils lisent des signaux, et l'affichage se met à jour de lui-même à la connexion comme à la déconnexion.

core/services/user.service.ts · memoire.service.ts

ACCÈS API

Rôle. Regroupent les appels HTTP vers `/api/users` et `/api/memoires`, et exposent des modèles typés.

Pourquoi. Un seul endroit à modifier quand un contrat d'API change, au lieu d'URLs dispersées dans les composants.

features/

ÉCRANS

Rôle. Un dossier par domaine fonctionnel : `auth` (connexion, inscription), `dashboard`, `memoires` (dépôt et consultation), `admin-memoires` (validation et rejet), `users`, `admin`, `profile`.

Pourquoi. Un découpage par fonctionnalité plutôt que par type de fichier : chaque écran reste lisible seul, et le chargement différé par route devient naturel.

shared/

COMMUN

Rôle. Le composant `shell` (mise en page, navigation) et les modèles `user.model.ts`, `memoire.model.ts`.

Pourquoi. Ce qui sert à plusieurs écrans vit ici, ce qui évite les imports croisés entre fonctionnalités.

environnements/

CONFIGURATION

Rôle. `environment.ts` vise l'API sur `localhost:5000` pour le développement ; `environment.prod.ts` utilise les chemins relatifs `/api` et `/auth` servis par la gateway.

Pourquoi. Le fichier de production est **indispensable au build** : `angular.json` le déclare dans `fileReplacements`, et sans lui aucune compilation n'aboutit. Il doit rester versionné et ne jamais contenir de secret, puisque tout ce qui entre dans un bundle est public.

assets/silent-check-sso.html

OIDC

Rôle. Page minimale servant de cible de redirection à la vérification de session silencieuse.

Pourquoi. Permet à keycloak-angular de tester la session dans une iframe sans recharger l'application.

5 Les fichiers de configuration

FICHIER	RÔLE ET BUT
<code>docker-compose.yml</code>	Décrit les dix services, le réseau commun, les volumes persistants et les dépendances de démarrage. Les valeurs sensibles passent par des variables d'environnement avec repli.
<code>.env.example</code>	Modèle des variables attendues (mots de passe, URL publique, realm). À copier en <code>.env</code> , qui est ignoré par git.
<code>gateway/nginx.conf</code>	Le contrat d'entrée : les cinq préfixes d'URL et leurs cibles internes.
<code>keycloak/realm-export.json</code>	Le realm en un fichier : rôles, clients, mappeurs, compte superadmin et droits du compte de service. Importé au premier démarrage.
<code>keycloak/themes/usermgmt/</code>	Thème de connexion aux couleurs du projet, pour que l'écran Keycloak ne rompe pas l'expérience.
<code>backend/appsettings.json</code>	Chaîne de connexion, adresses Keycloak (publique et interne), MinIO, Redis, niveaux de journalisation. Surchargé par les variables d'environnement en conteneur.
<code>prometheus.yml</code>	Définit la tâche de collecte <code>aspnetcore</code> visant <code>api:8080/metrics</code> toutes les 15 s.
<code>grafana/provisioning/</code>	Déclare la source Prometheus — avec un identifiant figé, car un tableau de bord la référence par cet identifiant — et le dossier des tableaux de bord.
<code>grafana/dashboards/</code>	Deux tableaux complémentaires : trafic HTTP brut d'un côté, compteurs métier et latence p95 de l'autre.

6 Ce que chaque composant attend des autres

Vue condensée des dépendances : utile pour savoir ce qui tombe quand un service manque.

COMPOSANT	DÉPEND DE	CONSÉQUENCE SI INDISPONIBLE
gateway	web, api, keycloak, grafana, minio	plus aucun accès : c'est la seule porte d'entrée
web	aucun (fichiers statiques)	l'interface ne se charge plus, l'API reste utilisable
api	postgres, keycloak, minio, redis	voir les lignes suivantes, selon le service manquant
keycloak	keycloak-db	plus de connexion possible, et l'API ne peut plus valider les jetons
postgres	—	toutes les routes métier échouent
minio	—	dépôt et téléchargement de PDF impossibles ; le reste fonctionne
redis	—	aucun effet fonctionnel aujourd'hui, mais la connexion est établie au démarrage du service de stockage
prometheus	api	perte des métriques ; l'application n'est pas affectée
grafana	prometheus	tableaux de bord vides

7 Points de vigilance

Secrets par défaut en clair

`docker-compose.yml` et `realm-export.json` contiennent des valeurs de repli (`admin/admin` , `usermgmt-backend-secret` , `minioadmin`). Acceptable en développement, à remplacer impérativement par un `.env` avant toute exposition réseau.

Le mot de passe du compte superadmin est temporaire

Le realm le marque `temporary` : Keycloak impose son changement à la première connexion. C'est voulu, mais cela empêche d'obtenir un jeton par mot de passe sans passer par le navigateur.

Aucun test automatisé

Le projet ne contient ni fichier `.spec.ts` , ni projet de test `.NET`. Toute vérification passe aujourd'hui par la compilation et l'exécution réelle de la pile. C'est le principal manque structurel.

Redis sans consommateur

La clé écrite après chaque dépôt n'est jamais relue. Soit lui donner un usage (cache de métadonnées, sessions), soit retirer la dépendance pour ne pas laisser croire à un mécanisme de cache actif.

KC_PROXY est déprécié

Keycloak 26 signale l'option au démarrage. Elle fonctionne encore, mais devra devenir `KC_PROXY_HEADERS` lors d'une prochaine montée de version.